

Building a Quantum Machine Learning library in myQLM

Report on an internship at Eviden (Atos Group)

Master in Quantum Information Science and Technology

Author: César Hernando de la Fuente

Company supervisor: Andrés Bravo Montes

University supervisor: Matthias Möller

September 17, 2025



**Universiteit
Leiden**

EVIDEN

Contents

1	Introduction	3
1.1	Quantum Machine Learning	3
1.2	Eviden Quantum	5
1.3	Problem statement	5
2	Methods	6
2.1	Quantum Support Vector Machine	6
2.1.1	Overview of the SVM algorithm	6
2.1.2	Application of a feature map	8
2.1.3	Going Quantum	8
2.1.4	Implementation for two datasets: wine type and breast cancer detection	9
2.2	Quantum-Classical Convolutional Neural Network	10
2.2.1	Classical Convolutional Neural Networks	10
2.2.2	Going Quantum	11
2.2.3	Implementation of QCCNN for the MNIST dataset	11
2.3	Quantum Circuit Born Machine	12
2.3.1	Unsupervised generative learning	12
2.3.2	Description of the algorithm	12
2.3.3	Implementation of QCBM for the bars and stripes dataset	13
3	Results and deliverables	14
3.1	Quantum Support Vector Machine	14
3.1.1	Wine dataset	14
3.1.2	Breast cancer detection dataset	15
3.2	Quantum-Classical Convolutional Neural Network	16
3.3	Quantum Circuit Born Machine	18
4	Conclusion and reflection	20

1 Introduction

This report presents the work and contributions I have made during my 10-week internship at Eviden, as part of my master's programme in Quantum Information Science and Technology. Eviden is a subsidiary of Atos focused on sustainable digital solutions in areas such as Cybersecurity, High performance computing, Artificial Intelligence and Quantum Computing. The main objective of the internship was to develop a Quantum Machine Learning (QML) library within Eviden's software development stack, myQLM. In the following, I will provide a brief overview of QML and then describe Eviden's Quantum business solutions. After this introduction, the problem statement of the project will be outlined, along its impact on the organization, and then, the methods used and results obtained during my internship will be explained in detail.

1.1 Quantum Machine Learning

It is undeniable that one of the most important technology advancements of this century is Machine Learning (ML), a subdiscipline of artificial intelligence and statistics in which patterns are derived from large amounts of data with the goal to make sense of previously unknown inputs [1]. Thus, it gives the computer the ability to learn without being explicitly programmed. There are different types of ML methods with different learning goals, such as image classification, speech recognition, clustering of complex data, compression of information, strategy optimization, text generation and many others [2]. Since the volume of globally stored data grows significantly every year, the pressure to find innovative approaches to machine learning is rising. Moreover, due to the extremely high interest in AI across numerous sectors, innovations in the field have an enormous economic impact [1].

Another popular area of research during the last years has been Quantum Computing, although it is still in an early development stage and far from having revolutionary industry applications. The basic elements of this computation paradigm are the quantum bits or qubits, which are quantum two-level systems that behave according to the laws of Quantum Physics. This theory, which was developed more than 100 years ago, describes the submacroscopic world (although emergent phenomena can lead to quantum effects in larger scales), and has been extensively tested. Qubits, in contrast to classical bits, can be in a superposition of states 0 and 1, and only when measured the qubit collapses randomly to one state or the other [3]. For an extensive introduction to Quantum Mechanics, these books are usually recommended [4, 5, 6]

Nowadays, there exist several platforms to realise qubits, such as superconducting qubits, spin qubits, trapped-ions, neutral atoms, photon qubits and many others. Regardless of the physical implementation, a qubit's state can be modified through single qubit quantum gates, which in practice are implemented in different ways depending on the qubit platform. Moreover, qubits can be entangled with each other via two (or more) qubit gates. Entanglement is a purely quantum phenomena that makes the probability of measuring a qubit in a certain state depend on the measurement outcome of another qubit, and has profound consequences in quantum information theory, communication and cryptography [7, 8].

These two phenomena, superposition and entanglement, are the heart of quantum computing, and allow

parallel processing of inputs. Together with a smart use of entangling and single qubit gates, custom measurement observables and some classical post-processing, quantum algorithms can be created. Currently, there exists a zoo of quantum algorithms which have been shown to have an advantage in computational complexity compared to their classical counterparts. Perhaps the most popular ones are Grover’s and Shor’s algorithms, but many others exist, which can be found in [9].

However, current quantum computers have several problems and limitations that hinder the implementation of quantum algorithms in industry. Nowadays, quantum computers are in the Noisy Intermediate-Scale Quantum (NISQ) era, since qubits have limited coherence times, as it is very challenging to isolate them from the environment, and gates have limited fidelity. Quantum error correction (QEC) is progressing, and prototypes of logical qubits have been demonstrated, but large-scale, fault-tolerant quantum computers are not yet available. Additionally, the leader companies have computers of 50-1000+ physical qubits, which are not enough to run quantum algorithms using QEC, as these require millions of qubits [10].

To circumvent these difficulties, different hybrid algorithms have been proposed, in which quantum computation is used as a subroutine of an algorithm executed in a classical computer. Perhaps the most popular ones are VQE (Variational Quantum Eigensolver), used for Quantum Chemistry and Condensed Matter Physics [11]; and QAOA (Quantum Approximate Optimization algorithm), used for combinatorial optimization [12]. Both of these algorithms use parametrized quantum circuits, in which some rotation gates’ angles are variational parameters. These parameters are then tweaked by a classical optimizer to minimize the expectation value of a Hamiltonian or a cost function.

Quantum Machine Learning algorithms also have this hybrid spirit, in which quantum computing is used for subroutines of the main algorithm. Similarly to classical ML, QML can be divided in three areas: supervised learning, unsupervised learning and reinforcement learning. In each of the categories, there exists an immense number of different algorithms with different purposes and architectures. In spite of their differences, all QML algorithms are designed to provide an advantage with respect to their classical counterparts. This advantage can refer to a reduction in computational complexity, a reduction in the training data required, a higher expressivity, or a different learning representation that may result in different solutions [13].

These advantages are generally theoretical, and not possible to obtain in practice due to the limitations of current quantum devices. Additionally, QML algorithms necessitate many iterations and consequently, many quantum circuit executions. Thus, using quantum computers in the cloud is not advantageous, being quantum emulators commonly employed to assess and validate QML algorithms. However, the computational complexity of simulating quantum circuits is exponential with respect to the number of qubits, and therefore, the datasets used in QML have a limited number of features.

At the present time, there are numerous frameworks to design and execute quantum algorithms in either real quantum devices or in quantum emulators, such as Qiskit [14], PennyLane [15], Cirq [16], or myQLM [17]. The latter was developed by Atos, the company where I did the internship, and it is the quantum software development stack (SDK) that I used to implement the quantum subroutines of the QML algorithms. In the following section, I will explain briefly the business endeavors of the department I was part of during the internship.

1.2 Eviden Quantum

Eviden offers different products and services within Quantum Computing. First of all, as mentioned early, it provides free access to myQLM, a Python package that allows students and developers to test their ideas directly from their laptops. In addition, it contains interoperability connectors with frameworks such as Qiskit, Cirq, ProjectQ or Forest, which gives flexibility to the user [18]. Finally, it can be used as a front-end to the Qaptiva 800, a quantum emulator that offers higher performances, larger simulation capabilities, being able to simulate up to 40 logical qubits, and advanced features like quantum circuit optimizers and noisy simulators [19].

Additionally, Eviden offers Qaptiva HPC, a licensed software that offers a full programming and emulation environment specifically designed for high-performance computing (HPC) environments. This innovative solution significantly boosts simulation capabilities, resulting in remarkable improvements in computing speed, with capacity limited only by the cluster memory. It is built in C++ with a Python API, enabling seamless integration and usability. Finally, it is designed for multi-node operation, being fully compatible with Slurm scheduling [20].

Some of the clients of Eviden in the past years have been the Jülich Supercomputing centre (Germany) [21], CESGA (Spain) [22], SCAYLE (Spain) [23], Hartree centre (UK) [24], GENCI (France) [23] and Université de Reims Champagne-Ardenne (France) [23]; and it has participated in big European projects such as HPCQS [25] and NEASQC [26].

Apart from selling the products mentioned before, assessing use cases and providing support, Eviden also delivers training workshops in universities. The workshops serve as an introduction to the fundamentals concepts of quantum computing, as well as some more advanced quantum algorithms. Furthermore, the workshops include applying the concepts learned with the help of Jupyter Notebook tutorials [27].

1.3 Problem statement

Quantum Machine Learning is a very popular area of research nowadays, in both academia and industry. Some of the giants of quantum computing such as IBM, Xanadu or Google have already developed a QML library, as well as several tutorials of QML algorithms. However, Eviden did not have entered yet into this field, even though they do have many quantum optimization classes implemented, as well as many ongoing research projects. Thus, I was appointed to develop a QML library in myQLM, together with Jupyter Notebook tutorials applying the algorithms to specific datasets.

This would bring value to Eviden in several ways. First of all, it would allow users to explore QML methods from a high-level perspective. Secondly, it would make myQLM more competitive with respect to other quantum SDKs. Thirdly, it would enable the delivery of more advanced training modules in educational quantum computing workshops. Fourthly, it would motivate the community, including researchers, to create and test their own QML routines, and develop demos which could be added to the official myQLM repository [28]. Finally, my work would serve to assess the performance of state of the art quantum machine learning algorithms.

2 Methods

There exists a large number of QML algorithms, each of them with multiple variants. In this project, due to the limited time and the need to balance the research work with other tasks, I decided to narrow my scope to three QML algorithms. The first one is a supervised binary classification algorithm: the Quantum Support Vector Machine. Secondly, we explored a hybrid approach to tackle a more difficult problem consisting of multi-class image classification: the Quantum-Classical Convolutional Neural Network. Finally, I wanted to delve into unsupervised and specifically, generative learning, and I found the Quantum Circuit Born Machine algorithm.

Together with the implementation of the algorithms using object-oriented programming for reusability, I wrote documentation files so that team members and users could understand and utilize the QML algorithms implemented. Moreover, I created Jupyter notebooks where I applied each algorithm to at least one dataset, and tested their performance.

The code, tutorials, analyses and documentation are available in the github repository:

<https://github.com/cesar-hernando/QML>.

2.1 Quantum Support Vector Machine

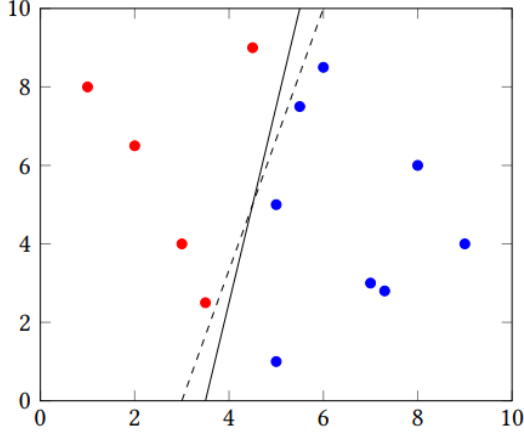
The quantum Support Vector Machine (QSVM) is a QML algorithm designed for binary classification, inspired by the classical Support Vector Machine (SVM). In the following, we will briefly review the SVM algorithm and then study its quantum version. After that, we will apply it to two datasets.

2.1.1 Overview of the SVM algorithm

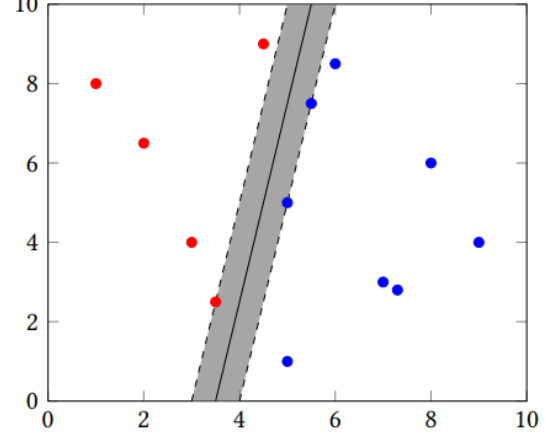
Let us suppose that we are given a set of points $x \in \mathbb{R}^2$ lying on a plane, each of them with a label $y \in \{0, 1\}$. The goal is to find a line that separates the points of different labels (see Figure 1a, taken from [29], where the color indicates the binary label). If such a line exists, which sometimes does not, there are infinite lines that do the job. However, some are better than others, as they have a greater margin from the closest points from each class.

So far we have introduced a linear classifier, but the SVM is more than that, it is a ML algorithm [2]. Thus, it does not only aim to classify the given points but tries to learn the underlying probability distribution $P(y|x)$ and, after training is completed, be able to classify correctly unseen points (hoping that they follow the same probability distribution). Therefore, the separating line is more likely to generalize to unseen data if it has a wider margin to the closest points in the training data.

The equation of a line can be expressed as $\mathbf{w} \cdot \mathbf{x} + b = 0$, where $\mathbf{w}$ is a vector normal to the line. We can also define the equation of a parallel line that passes through the closest point to the separating line, as well as the equation associated to the symmetric line (see Figure 1b). Normalizing these equations, they can be expressed as $\mathbf{w} \cdot \mathbf{x} + b = \pm 1$, where the distance between them can be shown to be $2/||\mathbf{w}||$. Hence, the goal



(a) Both lines (hyperplanes) separate the two categories, but the continuous line is closer to the data-points than the dashed line.



(b) The hyperplane that could have been returned by an SVM is represented by a black continuous line, and the lines in dashed lines are the equidistant parallel hyperplanes.

Figure 1: Linear classifier vs SVM. Both figures are taken from [29].

is to maximize this margin, i.e., solve the following optimization problem:

$$\text{Minimize } \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{subject to } y_j(\mathbf{w} \cdot \mathbf{x}_j + b) \geq 1, \quad \forall j \quad (1)$$

This optimization problem is known as the hard margin case, as no point is allowed to be misclassified or lie inside the margin. However, ML datasets usually contain noise and the goal of ML algorithms is to distinguish noise from important information. Thus, the previous constraint is relaxed, and the optimization problem that SVM actually solves is the following (we skip the derivation here, but it can be found in [30]):

$$\begin{aligned} &\text{Maximize } \sum_j \alpha_j - \frac{1}{2} \sum_{j,k} y_j y_k \alpha_j \alpha_k (\vec{x}_j \cdot \vec{x}_k), \\ &\text{subject to } 0 \leq \alpha_j \leq C, \quad \forall j, \quad \text{and } \sum_j \alpha_j y_j = 0. \end{aligned} \quad (2)$$

This is a convex quadratic programming problem, which can be solved using Sequential Minimal Optimization (SMO) [31]. Finally, we note that the SVM does not only apply to datasets with two features (or points in a plane), this was just done for visualization purposes, but can be applied to datasets with arbitrary number of features. Thus, in the general case, the separating line becomes a hyperplane, but the mathematics do not change.

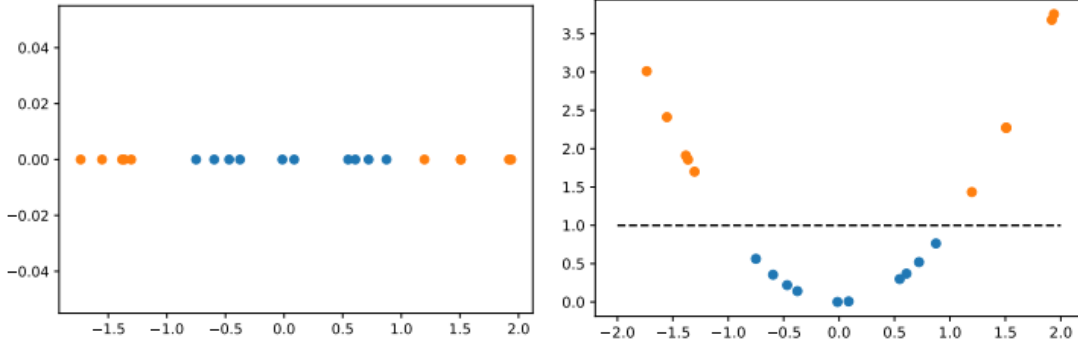


Figure 2: Example of the use of a feature map to make data linearly separable. These figure are taken from [29].

2.1.2 Application of a feature map

The algorithm defined so far can only classify data satisfactorily if it is linearly separable. This is rarely the case, especially for datasets with many features. There is a trick though, which consists on mapping the data to a higher dimensional space (known as feature map), and hope that in this space the data points are linearly separable (see Figure 2). This procedure is known as the Kernel trick in the literature, as $\mathbf{x}_j \cdot \mathbf{x}_k$ is replaced by $K(\mathbf{x}_j, \mathbf{x}_k) = \langle \phi(\mathbf{x}_j) | \phi(\mathbf{x}_k) \rangle$, where ϕ denotes the feature map. Applying the kernel for all the possible pairs of datapoints defines the kernel matrix.

2.1.3 Going Quantum

Having explained the classical SVM algorithm, we are ready to go Quantum. The idea is to use a parametrized quantum circuit as the feature map, and make use of the high dimensionality of the Hilbert space (2^n where n is the number of qubits). There are many different ways to map vectors to quantum states [29] but here we will explain and implement the angle encoding feature map and the ZZ feature map. On the one hand, angle encoding consists of, starting with n qubits (which must coincide with the number of features of the dataset) in state $|0\rangle$, applying a rotation gate to each qubit, where the angle for each gate is the value of the corresponding feature (after being normalized), as it is shown in Figure 3a. On the other hand, the ZZ feature map is a deeper circuit that includes CNOT gates, being its depth dependent of the number of qubits (quadratically). The specific description of this feature map can be found in the github or in [29]. It is depicted in Figure 3b. Thus, it can explore entangled states, so it has a higher expressivity than angle encoding.

Since what we need is the kernel matrix (composed by the inner product of vectors in feature space), we need to calculate the following quantities:

$$K(\vec{x}_j, \vec{x}_k) = \langle \phi(\vec{x}_j), \phi(\vec{x}_k) \rangle = |\langle 0 | \phi^\dagger(\vec{x}_j) \phi(\vec{x}_k) | 0 \rangle|^2 \quad (3)$$

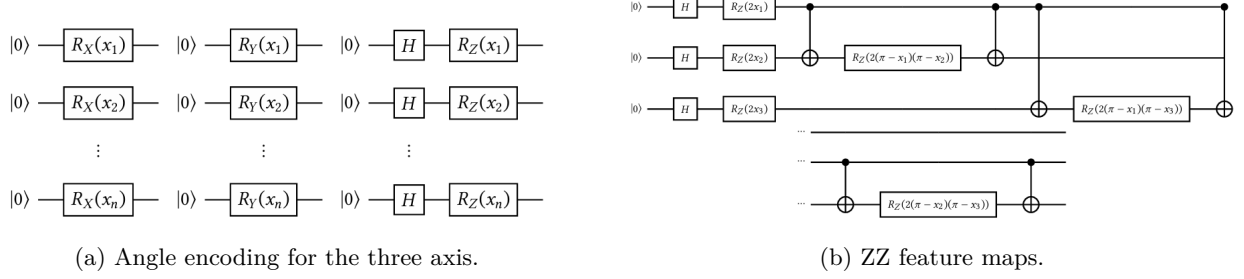


Figure 3: Quantum circuits corresponding to the angle encoding and ZZ feature maps.

Hence, after applying the feature map to the first vector, we need to apply the inverse feature map (replacing every gate by its inverse) to the second vector. Finally, we will execute the quantum circuit n_{shots} times (measuring in the computational basis) and estimate the probability of measuring all qubits in the $|0\rangle$ state.

Having calculated the quantum kernel matrix, which requires $n_{shots} \times n_{samples}^2$ runs of the quantum circuit (with number of qubits = number of features), we can apply the "normal" SVC and SMO to find the optimal classifying hyperplane. After the optimal normal vector and bias of the hyperplane is found, i.e. training is complete, the algorithm can predict the label of new data points by calculating: $sign(\mathbf{w} \cdot \mathbf{x}_{new} + b^*)$.

2.1.4 Implementation for two datasets: wine type and breast cancer detection

We applied the QSVM to two datasets. The first one is a classic and very easy multi-class classification dataset. In order to make it binary, we only used the first 130 rows of the dataset. This dataset has 13 features, and other properties can be found here: https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_wine.html. The second dataset is for breast cancer detection. It has 569 samples and 30 features, and more information can be found here: https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_breast_cancer.html.

We now explain the pipeline for applying the QSVM to these datasets. First, we split the data into training and test sets so that the performance of the model is measured on unseen data. Secondly, we scale the features (between 0 and 1, although could also do it between 0 and 2π) using only the training data to prevent data leakage.

Thirdly, QML routines using simulators are usually very slow, due to the large number of runs required for learning. Additionally, the number of qubits, which for both angle encoding and ZZ feature map, is equal to the number of features, and must not be too high. Therefore, to reduce the number of features we apply a commonly used dimensionality reduction technique called Principal Component Analysis (PCA) [32]. This method first finds the directions of maximum variance (principal components) and then performs a linear transformation, or change of basis, so that the feature space (rows of the dataset) is expressed in the basis of principal components. Finally, the columns associated with the directions of least variance are removed, as they contain the least amount of information. In the section 3.1, we analyse the accuracy of the algorithm for different values of the number of PCA components, which determines the number of transformed features

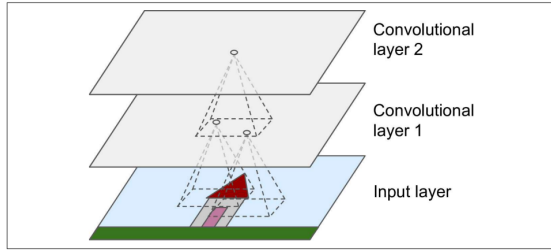
that are kept. We also vary the feature map, comparing ZZ feature map and angle encoding.

2.2 Quantum-Classical Convolutional Neural Network

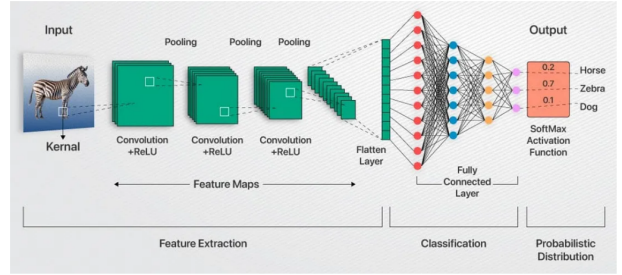
The Quantum-Classical Convolutional Neural Network (QCCNN) is a quantum-enhanced supervised learning method for image classification into multiple classes. It combines a quantum convolutional layer with a classical CNN. Before explaining this method, we will briefly introduce convolutional neural networks.

2.2.1 Classical Convolutional Neural Networks

Convolutional Neural Networks (CNN) emerged from the study of the brain's visual cortex, and they have been used in image recognition since the 1980s. Thanks to the increase in computational power and their training with backpropagation, they have been able to achieve superhuman performance in complex visual tasks such as image search services, automatic video classification systems, and even self-driving cars [33]. In contrast to fully-connected neural networks (FCNN), where images have to be flattened to be processed and every neuron in the first layer is connected to all pixels in the image, in CNN each neuron is only connected to neurons in its receptive field, as shown in Figure 4a [33].



(a) CNN layers with local receptive fields. This figure is taken from [33]



(b) Architecture of Convolutional Neural Networks

Figure 4: Graphical representation of Convolutional Neural Networks

The fundamental building block of a CNN is the convolutional layer, which consists of applying a convolution between the image (or previous layer) with a kernel. The kernel is just a matrix which can have several filters (making it a rank 3 tensor), and the convolution consists of sliding this kernel through the image and multiplying element by element, adding the results and applying an activation function such as ReLU. The output is thus another image of reduced size, but usually with more channels. The other block of CNNs is the pooling layer, which takes square sections of the image (of a certain size) and either outputs the maximum (MaxPooling) or average (AveragePooling). In this way, the dimensionality of the image is reduced. Finally, after applying several convolutional and pooling layers, the image is flattened and a FCNN is applied to classify the image. This set up is shown in Figure 4b. This architecture allows the network to concentrate on small low-level features in the first hidden layers, then assemble them into larger higher-level features in the next hidden layer, and so on, which is why this type of NN works so well for image recognition [33].

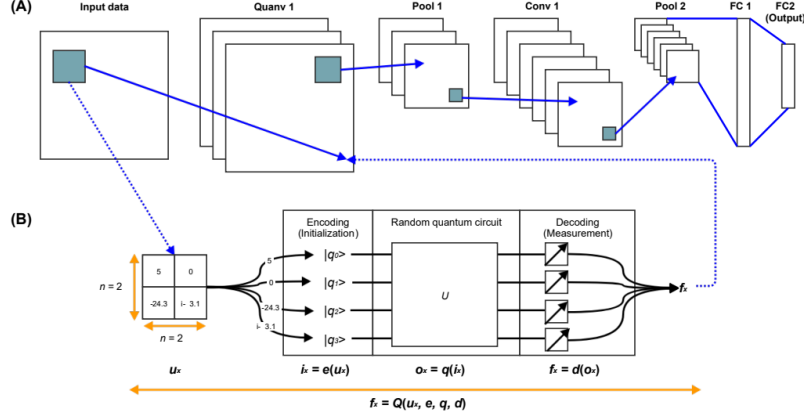


Figure 5: Architecture of Quantum-Classical Convolutional Neural Networks [36]

2.2.2 Going Quantum

There are many different types of architecture that combine CNNs with quantum circuits. Some of them use a single quantum circuit to process the whole image, mapping pixels to qubits through a feature map, and using variational parameters to simulate the weights. These are called QCNNs if the quantum circuits are translationally invariant. Some papers describing this type of Quantum CNNs are [34] and [35], but they have the inconvenience that with the actual NISQ devices and simulators, which can process a small number of qubits, only very small images and toy datasets can be implemented. For instance, the MNIST dataset, which has images of 784 pixels, around 784 qubits are needed.

Thus, we focused on hybrid Quantum-Classical Convolutional Neural Network (QCCNN), termed as Quantvolutional Neural Network in [36] and [37]. The quanvolutional layer consists of, instead of multiplying small rectangles of the image with a kernel, applying small quantum circuits of 4 (2x2) or 9 (3x3) qubits, which first map the pixel values of the rectangle to quantum states through a feature map, and then apply an ansatz (which is random and fixed in the paper). In [36], $\langle Z \rangle$ is measured for each qubit, and each of the results is used as the corresponding pixel value of each of the channels of the next layer. After applying one or several of these quanvolutional layers, classic convolutional and pooling layers are applied, followed by a FCNN. This architecture is shown in Figure 5 [37].

In our implementation, we substitute the random circuit with a variational one. For training, the logic works as follows: first, we initialize the quantum variational parameters randomly and train the classical neural network with backpropagation and Adam; and then, with the trained classical model fixed, we train the variational angles of the quanvolutional layer.

2.2.3 Implementation of QCCNN for the MNIST dataset

MNIST is a commonly used dataset for benchmarking that contains 28x28 gray scale images of hand-written digits, along with a label of the digit they represent. The first step in the implementation consists of selecting

the training and test images (300 and 75), and normalizing the pixels between 0 and 1. Then we define the quantum and classical models. The quantum layer uses kernels of 4 qubits, angle encoding feature map, the hardware-efficient ansatz [38], the magnetization of each qubit as observables, and no shot noise. Regarding the CNN, the first convolutional layer has 15 filters, each of size 2x2 and ReLu activation function. Then, we use MaxPooling of size 2x2, after which the tensor is flattened and two fully connected layers are applied, with 64 and 10 neurons, respectively. The first one uses ReLU but the last one uses SoftMax, to output the probability of the image corresponding to each of the digits from 0 to 9.

For training the CNN, we use backpropagation with Adam optimizer (learning rate of 0.001), the sparse categorical cross entropy loss function, batch size of 16 and 60 epochs. We apply the CNN to both the original images and the images after the quantum preprocessing and compare the test loss and accuracy for both of them.

Additionally, as an add-on to the original implementation described in [36], we fix the CNN and tweak the angles in the quantum gates to try to improve the accuracy further. We utilized the COBYLA optimizer (with 30 iterations, limited by computational time), which does not require gradients, as integrating automatic differentiation of quantum circuits in myQLM would have required more than 10 weeks probably. For this, I used 300 additional training images, and minimized the training loss. After this, the hybrid model is tested again. The results will be shown and discussed in section 3.2.

2.3 Quantum Circuit Born Machine

The Quantum Circuit Born Machine (QCBM) is an unsupervised generative machine learning algorithm that leverages quantum circuits and Born’s postulate to encode probability distributions. Before delving into the workflow of the algorithm, we will introduce the unsupervised generative learning paradigm.

2.3.1 Unsupervised generative learning

In contrast to supervised learning, unsupervised models only have access to datasets of inputs with no corresponding labels. Therefore, their goal is not to learn to predict the label y associated to input vectors x , or in the probability formalism the conditional probability $P(y|x)$, but to learn the underlying probability distribution $P(x)$ of the dataset. There are different types of unsupervised models, such as discriminative (clustering, e.g. K-means or DBSCAN), dimensionality reduction (PCA, Autoencoders, etc.), and generative (GANs, Variational Autoencoders, Diffusion models, etc.). Here we focus on generative models, where after learning $P(x)$, the goal is to generate samples resembling the original data [33].

2.3.2 Description of the algorithm

QCBM is a generative model that encodes the probability distribution of classical data as a variational quantum circuit, and it is based on Born’s postulate of Quantum Mechanics, which states that the probability of obtaining the bitstring x as a result of measuring the quantum state $|\psi_\theta\rangle$ is $|\langle x|\psi_\theta\rangle|^2$ [39]. In contrast

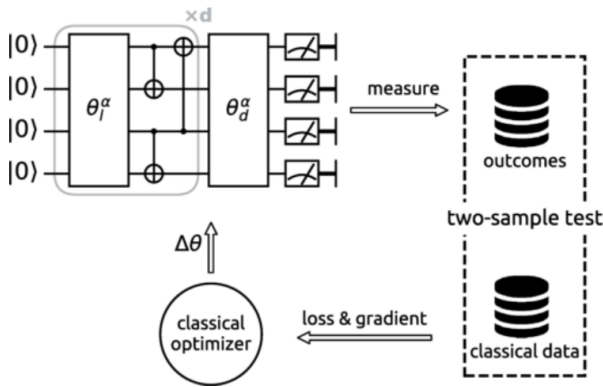
to other methods such as Restricted Boltzmann machines, QCBM samples efficiently, as it just requires executing the quantum circuit.

Regarding the training process, the variational circuit is initialized with random parameters, and for each iteration it produces a specified number of samples and a loss function is evaluated, which measures the distance between the target distribution and the model distribution. The QCBM in [39] employs the squared mean discrepancy loss [40] with a Radial Basis Function (RBF) kernel [41]. Similarly to supervised QNNs, the gradient of the loss function with respect to the variational parameters can be evaluated exactly via the Parameter Shift Rule [39]. Thus, we can use gradient-based optimizers such as Adam or L-BFGS-B [6]. The training procedure is sketched in Figure 6a. For a more detailed explanation of the training procedure, we recommend reading [39].

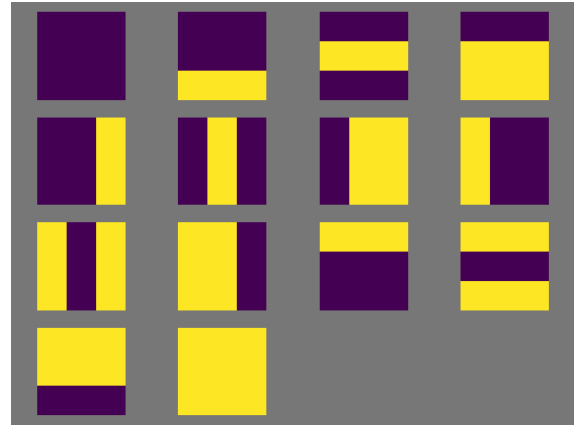
After the optimal parameters are found and training is finished, we will generate samples from the circuit and analyse their validity. Apart from the precision metric, which measures the rate of generating valid patterns, there are other metrics that measure generalization, which refers to not only considering the validity criterium but also whether the generated sample was seen in training. However, since our dataset is very small, it does not make sense to study its generalizability. For a study of QCBM generalization capabilities, read the paper [42].

2.3.3 Implementation of QCBM for the bars and stripes dataset

In this project and in the Jupyter notebook tutorial, we implemented a QCBM algorithm to a dataset of Bars and Stripes. This dataset is composed by binary matrices $N \times N$ that form a bar of stripe pattern. Specifically, a matrix is considered a bar if for every column, all its elements are the same. Similarly, a matrix is considered a stripe if for every row, all the elements are the same. Here we consider 3×3 matrices, for which there are 14 valid solutions. Thus, these 14 matrices will compose the training dataset of the QCBM, and are depicted in Figure 6b.



(a) Diagram of the QCBM training process. This figure is taken from [39].



(b) Valid solutions in the bars and stripes dataset.

Figure 6: Quantum Circuit Born Machine and Bars and Stripes dataset.

Regarding the hyperparameters used for the QCBM, we use circuits of 9 qubits (as the images have 9 pixels). The ansatz has 8 block, we use 20 000 shots for each two-sample test (loss function evaluation), Adam optimizer with learning rate of 0.1 and 70 iterations, and for the RBF kernel we choose three gaussians with standard deviations 0.25, 0.5 and 1. After training, we generate 200 000 samples to test the performance of the algorithm. The results will be discussed in section 3.3.

3 Results and deliverables

In this section, I will present the results obtained for the three QML methods. Before that, you can find all the deliverables I worked on in the github repository <https://github.com/cesar-hernando/QML>. It includes the python file myqml.py (more than 1000 lines), which contains the definitions of the classes of the three methods, as well as some helper classes with feature maps that are used in many QML algorithms. Moreover, I worked on Jupyter Notebook tutorials to help users and developers to understand the QML methods and test them themselves. Furthermore, for each method there is a Jupyter notebook with its corresponding documentation. Finally, the repository includes all the papers I used in my research and the power points that I used for my presentation in the company.

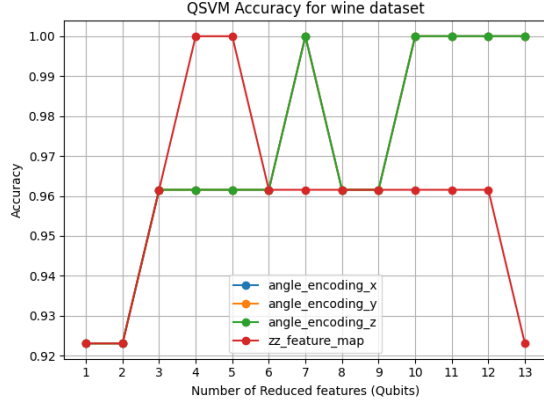
3.1 Quantum Support Vector Machine

This algorithm was applied to two datasets of increasing complexity, as explained in section 2.1. In the following, we will discuss the results obtained for each of the datasets.

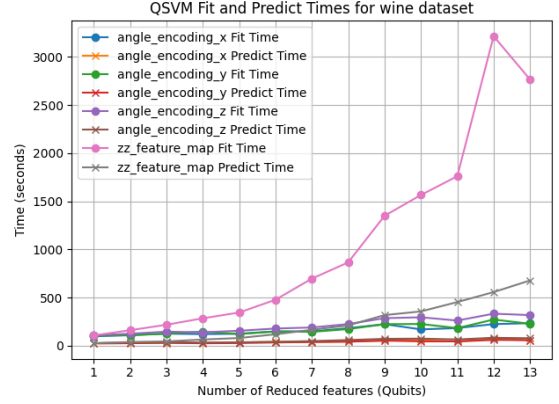
3.1.1 Wine dataset

For this dataset, we varied the number of PCA components from 1 to 13 (no PCA applied), as well as the feature map (angle encoding using R_X, R_Y and R_Z gates, and the ZZ feature map). The accuracy on the test data for each case is shown in Figure 7a, while Figure 7b depicts the training and prediction time.

On the one hand, Figure 7a shows that all feature maps achieve an accuracy higher than 92% even when only the one PCA component (corresponding to the direction of maximum variance) is selected. This means that most of the features of the dataset do not affect that much the wine type label and therefore the dataset is very easy to learn, as described in its description (link in section 2.1.4). For 4 and 5 PCA components, the ZZ feature map reaches to an accuracy of 100%, whereas angle encoding (regardless of the axis) achieves 96%. However, for 10-13 components (features and qubits), angle encoding outperforms ZZ feature map, achieving 100% accuracy. The fact that sometimes the accuracy is higher with less PCA components could be caused by two factors. The first one is that by performing a more aggressive PCA, we could be getting rid of directions containing noise, and therefore making the dataset easier to learn. The second one is that increasing the PCA components, increases the qubits and therefore the Hilbert space. Thus, the expressivity of the model could be too high compared to the training samples, leading to overfitting. That could be the



(a) Test accuracy.



(b) Training and prediction times.

Figure 7: Accuracy (a) and execution times (b) of the QSVN applied to the wine dataset for different feature maps and PCA components (reduced number of features).

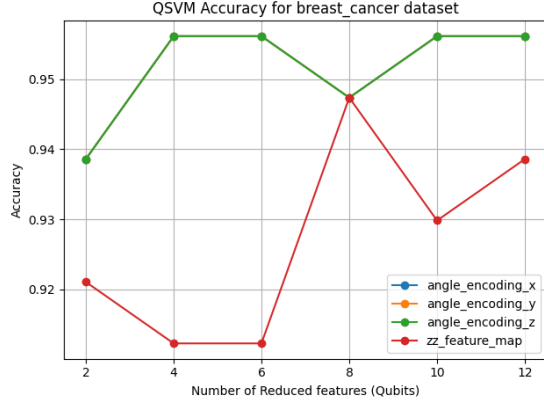
reason why the accuracy of the ZZ feature map drops for high number of features, while the angle encoding does not, as the ZZ feature map has a higher expressivity.

On the other hand, Figure 8b illustrates the training and prediction times. We note that the training time is always higher than the prediction time, as it should be. Moreover, we observe that using the ZZ feature map leads to a significantly higher execution times, especially for training. This makes sense, as its depth grows quadratically with the number of qubits. Additionally, we notice that the angle encoding in the Z axis requires a slightly higher training and prediction time, as it includes Hadamard gates before the rotation gates.

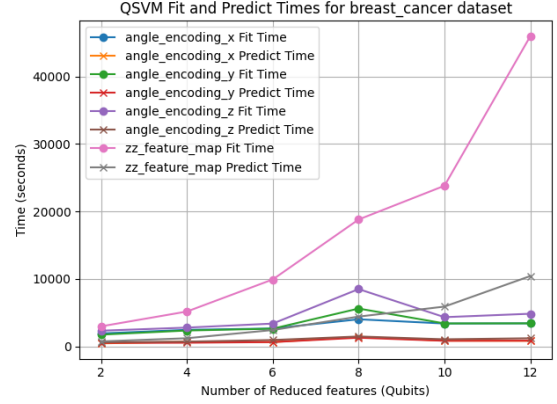
In short, for this dataset, there are two optimal options: either using ZZ feature map and 4 PCA components, or angle encoding (in x or y) and 10 PCA components.

3.1.2 Breast cancer detection dataset

As this dataset has more features and samples, we varied the PCA components from 2 to 12 in steps of two. Figure 8a shows the test accuracy. We observe that angle encoding (all axis overlap) consistently outperforms the ZZ feature map, achieving an accuracy of almost 96%. The fact that 4 PCA components is enough to achieve this accuracy proves that this is also a toy dataset, although more complex than the previous one. Regarding training and prediction times, Figure 8b also illustrates that the ZZ feature map is more expensive. Moreover, comparing the times with the previous dataset, we see that for 12 qubits and ZZ feature map, training lasts almost 50000 s (almost 14 hours), whereas the training of the previous dataset in this case was around 3000s (50 mins). This difference makes sense, as the number of circuit executions (and kernel matrix elements) scales quadratically with the number of samples, and the breast cancer dataset has around 4 times more samples. Thus, it makes sense that the training of the breast cancer dataset takes around 16 times longer.



(a) Test accuracy.



(b) Training and prediction times.

Figure 8: Accuracy (a) and execution times (b) of the QSVN applied to the breast cancer dataset for different feature maps and PCA components (reduced number of features).

3.2 Quantum-Classical Convolutional Neural Network

As described in section 2.2, the training process in the original paper [36] consists of apply the quanvolutional layer with random angles as a preprocessing step (see Figure 9 and then train the classical CNN as usual. In Figure 10a, we compare the accuracy (both training and validation/test) with and without the quantum layer with respect to the number of epochs. In Figure 10b, the training and test loss for both models are shown. Looking first at the accuracy graph, we deduce that both models memorize the data perfectly, as the training accuracy reaches 100%, but the quantum model takes more epochs to do it. This is reasonable, as the quantum model is slightly more complex, as the input images have four channels instead of one. From the moment the training accuracy of the classical model reaches to 100%, the validation accuracy reaches to a plateau at around 77%, and it slightly overfits. In contrast, the quantum model validation accuracy continuous increasing up to 84%. Thus, adding the quanvolutional layer increases the validation accuracy 7%.

This accuracy could be further increased by adding more quantum or classical layers. However, adding more quantum layers implies a very large pre-processing time., and adding more classical layers implies reducing the importance of the quantum layer, which goes against the goal of studying the effect of the quantum layer. In order to see what accuracy could be achieved classically in a reasonable amount of time, and without high performance computers or GPUs but with a normal laptop, we created a CNN with two convolutional layers: one with 50 filters and kernel size 2x2, and the other with 64 filters and kernel size 5x5. After each of them we apply MaxPooling of size 2x2. After this, we flatten the tensors, apply a FCNN of 64 neuros, a drop-out layer with 0.2 probability of a staying off during training. Finally, the output layer has 10 neurons naturally. With this model, a test accuracy of 96% was achieved. Including more training examples and more hidden layers can further increase the accuracy, but it requires more computational resources.

As an additional experiment, I thought it would be a good idea to also tweak the angles of the quantum layer, keeping the CNN parameter fixed. Since the training accuracy already reached to 100%, additional

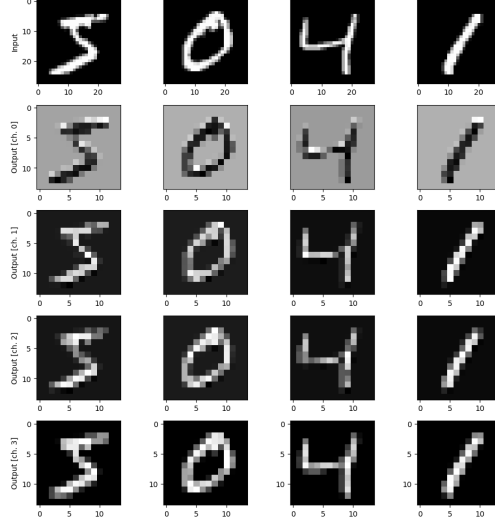


Figure 9: The first row shows some examples of the original images of the MNIST datasets. The other 4 rows shows 4 channels of the transformed images after the quanvolutional layer.

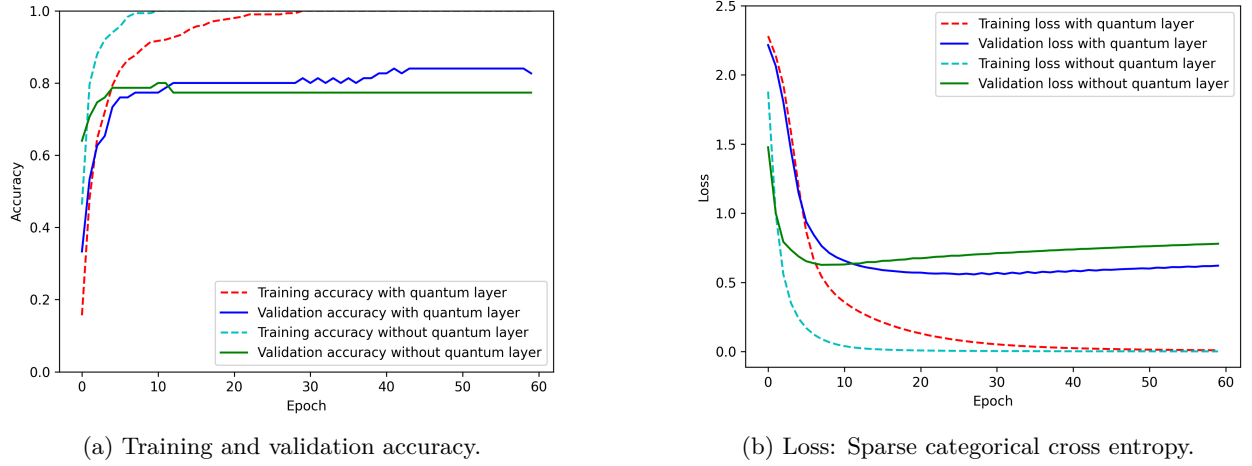


Figure 10: Comparison of performance when the quanvolutional layer is applied against when it is not applied.

images should be used for the quantum training. As mentioned in section 2.2, we used Cobyla optimizer. The evolution of the accuracy with the number of epochs is shown in Figure 11. We deduce that the model is not learning, as the accuracy is not increasing but fluctuating. The reason of this failure could be that changing the quantum ansatz requires tuning the classical model too. Thus, we should optimize the angles of the ansatz and the weights of the classical model together. This would require integrating automatic differentiation of quantum circuits in myQLM with tensorflow Keras. Unfortunately, this was out the scope of my 10-weeks internship.

Although this last step that I tried failed, this illustrates how many times in research trying something new does not work. Instead of seeing this a a failure, it just indicates that is not the path to follow and that

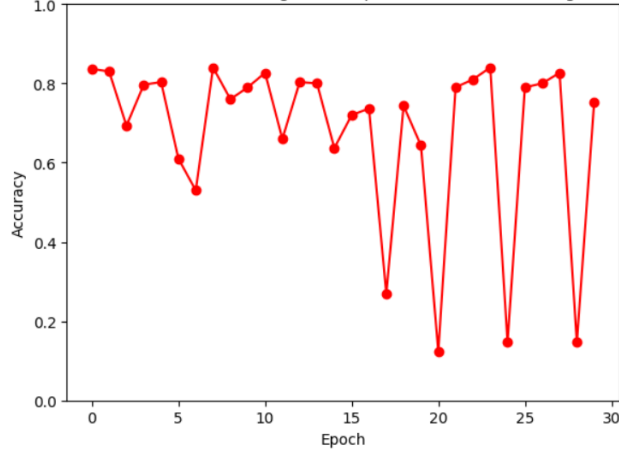


Figure 11: Evolution of the training accuracy with the number of epochs, when the angles of the quantum layer are optimized using Cobyala.

future research should go in a different direction.

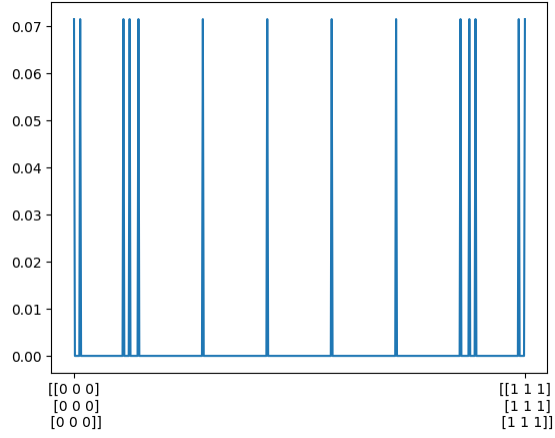
3.3 Quantum Circuit Born Machine

Using the hyperparameters described in section 2.3, we trained the QCBM to learn the bars and stripes patterns. Figure 13a shows the evolution of the training loss as a function of the number of epochs (or optimization steps). The graph shows a clear decrease of the loss function, reaching to a plateau in the end, meaning that the model is learning the desired distribution. This desired distribution that we want to learn is represented in Figure 12a

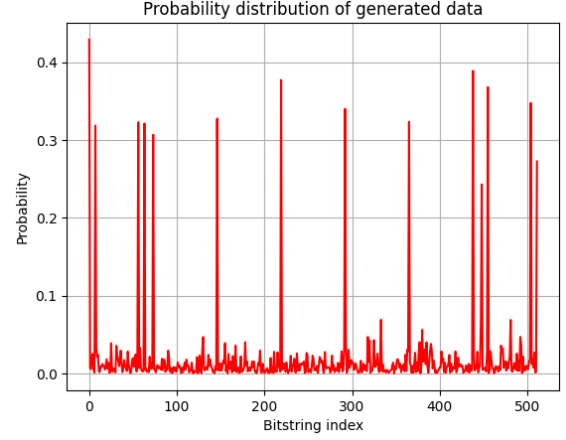
After training, we generated 200 000 samples. Figure 12b depicts the probability distribution of the generated samples. Comparing to the distribution of the valid solutions (see Figure 12a, we notice that the locations of the peaks match very well, although the height of the peaks is not uniform and there is some noise in between peaks. Overall, the QCBM learnt the distribution quite well, taking into account that we generated 200 000 samples. Increasing this number should improve further the performance.

Figure 13b illustrates some examples of the generated patterns. We see that many of the patterns are not valid, but most of them have only one or two pixels off. In order to quantify this, we calculated two metrics, the precision, defined as the rate of valid generated matrices; and the average cost, defined as the average number of pixels that should be flipped so that the matrix is valid (bar or stripe). Regarding the precision, we obtained 46.9% which is not that high, but still proves that the model is learning relevant patterns, as there are only 14 solutions and $2^9 = 512$ possibilities. In order to improve the precision, we should increase the number of shots, and fine tune the hyperparameters of the model. However, the average cost obtained is only 1.19, which implies that most invalid matrices are very close to being valid (usually by one pixel only).

So far we have actually studied the memorization capabilities of the QCBM, as we have not tested if the model can generate valid solutions that have not been seen during training. We tried to test the generalization



(a) Training data or valid solutions.

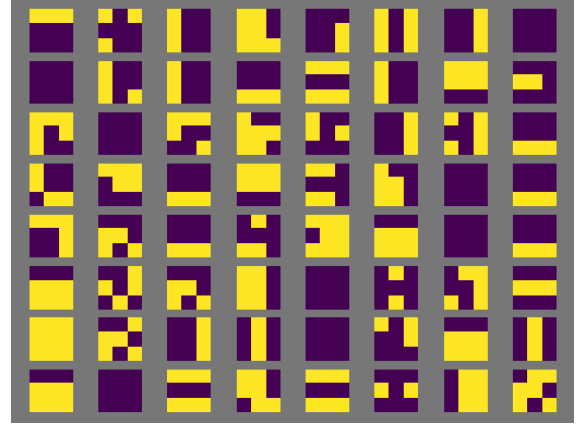


(b) Generated samples.

Figure 12: Probability distributions of training data (valid solutions) and of generated samples.



(a) Evolution of the MMD training loss with the number of epochs (optimization steps).



(b) Examples of the generated samples after training is completed.

Figure 13: Training and output of the Quantum Circuit Born Machine.

capabilities with these dataset, but the results were not satisfactory due to the limited number of valid solutions. In the future, another dataset with a higher number of valid solutions could be used. Then, the generalization metrics described in [42] could be assessed (fidelity, rate and coverage).

4 Conclusion and reflection

In conclusion, during my 10-weeks internship I implemented three Quantum Machine Learning algorithms in myQLM, Eviden's quantum software stack. I applied these algorithms to specific datasets and tested their performance. The results pointed out that although the three QML algorithms were correctly implemented, this area is still far from being useful for real datasets, due to its high computational time when running in simulators. However, it is still important to design new QML algorithms for the future, when the quantum hardware is more mature. In this scenario, QML algorithms could provide an advantage in terms of a reduction in computational complexity, a reduction in the training data required, a higher expressivity, or a different learning representation that may result in different solutions.

However, the main goal of this internship was not assessing QML algorithms performance, but creating Jupyter Notebooks tutorials for users and developers, in order to help them understand this new popular area of research. This task was completed satisfactorily. Moreover, I created documentation so that someone else could continue my work in the future, scaling the library and tutorials.

Although the QML library was my main task, I also provided value for the company with different tasks. Specifically, I worked together with another intern on a study of the main quantum simulators integrable in HPC clusters. We compared their different characteristics, such as CPU multi-core and multi-node compatibility, GPU acceleration, job scheduler integration, support service, and many others. This study was useful to the department in their write-up of a small section of a research paper. Additionally, I helped another intern of the team with his project on hyperparameter tuning of the Simulated Quantum Annealing (SQA) algorithm, implementing a Bayesian Optimization routine.

Reflecting back about my internship, I strongly believe it was a very valuable experience. First of all, I delved into the topic of Quantum Machine Learning, which I knew close to nothing in the beginning, and even learnt new classical Machine Learning algorithms, optimizers, and methods such as PCA. Secondly, I further improved my Python object-oriented programming (OOP) skills, learning useful concepts for package development such as abstract classes, overriding and inheritance. Thirdly, thanks to the interaction with my team members, I discovered which business activities the company works on, and that there is a lot of work to gain projects, including being in contact with clients and thinking strategically. Finally, through the progress presentations to the team, I improved my ability to explain concepts and results clearly, as well as to decide which information is relevant to communicate depending on the audience.

References

- [1] Maria Schuld, Ilya Sinayskiy, and Francesco Petruccione. “An introduction to quantum machine learning”. In: *Contemporary Physics* 56.2 (2015), pp. 172–185.
- [2] Christopher M Bishop and Nasser M Nasrabadi. *Pattern recognition and machine learning*. Vol. 4. 4. Springer, 2006.
- [3] N David Mermin. *Quantum computer science: an introduction*. Cambridge University Press, 2007.
- [4] David J Griffiths and Darrell F Schroeter. *Introduction to quantum mechanics*. Cambridge university press, 2018.
- [5] Ramamurti Shankar. *Principles of quantum mechanics*. Springer Science & Business Media, 2012.
- [6] Jun John Sakurai and Jim Napolitano. *Modern quantum mechanics*. Cambridge University Press, 2020.
- [7] Michael A Nielsen and Isaac L Chuang. *Quantum computation and quantum information*. Cambridge university press, 2010.
- [8] Thomas Vidick and Stephanie Wehner. *Introduction to quantum cryptography*. Cambridge University Press, 2023.
- [9] Stephen Jordan. *Quantum Algorithm Zoo* — *quantumalgorithmzoo.org*. <https://quantumalgorithmzoo.org/>. [Accessed 31-08-2025].
- [10] John Preskill. “Quantum computing in the NISQ era and beyond”. In: *Quantum* 2 (2018), p. 79.
- [11] Dmitry A Fedorov et al. “VQE method: a short survey and recent developments”. In: *Materials Theory* 6.1 (2022), p. 2.
- [12] Jaeho Choi and Joongheon Kim. “A tutorial on quantum approximate optimization algorithm (QAOA): Fundamentals and applications”. In: *2019 international conference on information and communication technology convergence (ICTC)*. IEEE. 2019, pp. 138–142.
- [13] José D Martín-Guerrero and Lucas Lamata. “Quantum machine learning: A tutorial”. In: *Neurocomputing* 470 (2022), pp. 457–461.
- [14] Ali Javadi-Abhari et al. “Quantum computing with Qiskit”. In: *arXiv preprint arXiv:2405.08810* (2024).
- [15] Ville Bergholm et al. “PennyLane: Automatic differentiation of hybrid quantum-classical computations”. In: *arXiv preprint arXiv:1811.04968* (2018).
- [16] Victory Omole et al. “Cirq: A Python framework for creating, editing, and invoking quantum circuits”. In: *URL <https://github.com/quantumlib/Cirq>* (2020).
- [17] Eviden. *myQLM – Quantum Python Package*. <https://myqlm.github.io/>. Accessed: 2025-09-05. 2025.
- [18] Eviden. *Quantum Computing Solutions* — Eviden. <https://eviden.com/solutions/quantum-computing/>. Accessed: 2025-09-05. 2025.

- [19] Eviden. *Qaptiva 800 Quantum Emulation Appliance — Brochure (EN, 2025)*. <https://eviden.com/wp-content/uploads/2025/08/Eviden-Qaptiva800-brochure-2025-EN.pdf>. Accessed: 2025-09-05. 2025.
- [20] Eviden. *Qaptiva HPC: Enabling Quantum Emulation in High-Performance Computing*. <https://eviden.com/wp-content/uploads/2024/12/Eviden-Qaptiva-HPC.pdf>. Accessed: 2025-09-07. 2024.
- [21] Eviden. *Eviden Provides the Modular Data Center to Host Europe’s First Exascale Supercomputer*. <https://eviden.com/insights/press-releases/eviden-provides-the-modular-data-center-to-host-europes-first-exascale-supercomputer/>. Press release. Accessed: 2025-09-07. Jan. 2024.
- [22] Centro de Supercomputación de Galicia (CESGA). *Quantum Infrastructure at CESGA*. <https://www.cesga.es/en/infrastructures/quantum/>. Accessed: 2025-09-07. CESGA, 2025.
- [23] Eviden (Atos Group). *Qaptiva Quantum Emulation Appliance*. <https://eviden.com/solutions/quantum-computing/qaptiva-quantum-emulation-appliance/>. Accessed: 2025-09-07. Eviden, 2025.
- [24] Eviden. *Hartree Centre Prepares for the Coming Quantum Revolution*. Accessed: 2025-09-07. 2025.
- [25] HPCQS Consortium. *HPCQS: High Performance Computer – Quantum Simulator Hybrid*. <https://www.hpcqs.eu/>. Accessed: 2025-09-07. 2025.
- [26] NEASQC. *NEASQC Project Website*. <https://www.neasqc.eu/>. Accessed: 2025-09-12.
- [27] Eviden. *Quantum Consulting, Training and Support*. <https://eviden.com/solutions/quantum-computing/quantum-consulting-training-and-support/>. Accessed: 2025-09-07. 2024.
- [28] myQLM. *myQLM — GitHub organization*. <https://github.com/myQLM>. GitHub. Accessed: 2025-09-12.
- [29] Elias F Combarro, Alberto Di Meglio Samuel González-Castillo, and Alberto Di Meglio. *A practical guide to quantum machine learning and quantum optimization*. Packt Publishing, 2023.
- [30] Yaser S Abu-Mostafa, Malik Magdon-Ismail, and Hsuan-Tien Lin. *Learning from data*. Vol. 4. AML-Book New York, 2012.
- [31] John Platt. “Sequential minimal optimization: A fast algorithm for training support vector machines”. In: (1998).
- [32] Jonathon Shlens. “A tutorial on principal component analysis”. In: *arXiv preprint arXiv:1404.1100* (2014).
- [33] Aurélien Géron. *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow*. ” O’Reilly Media, Inc.”, 2022.
- [34] Iris Cong, Soonwon Choi, and Mikhail D Lukin. “Quantum convolutional neural networks”. In: *Nature Physics* 15.12 (2019), pp. 1273–1278.
- [35] Tak Hur, Leeseok Kim, and Daniel K Park. “Quantum convolutional neural network for classical data classification”. In: *Quantum Machine Intelligence* 4.1 (2022), p. 3.
- [36] Maxwell Henderson et al. “Quantum convolutional neural networks: powering image recognition with quantum circuits”. In: *Quantum Machine Intelligence* 2.1 (2020), p. 2.

- [37] Nadine Matondo-Mvula and Khaled Elleithy. “Breast cancer detection with quanvolutional neural networks”. In: *Entropy* 26.8 (2024), p. 630.
- [38] Oleksandr Kyriienko, Annie E Paine, and Vincent E Elfving. “Solving nonlinear differential equations with differentiable quantum circuits”. In: *Physical Review A* 103.5 (2021), p. 052416.
- [39] Jin-Guo Liu and Lei Wang. “Differentiable learning of quantum circuit born machines”. In: *Physical Review A* 98.6 (2018), p. 062324.
- [40] Andrew Alden, Blanka Horvath, and Zacharia Issa. “Signature Maximum Mean Discrepancy Two-Sample Statistical Tests”. In: *arXiv preprint arXiv:2506.01718* (2025).
- [41] K Mike Tao. “A closer look at the radial basis function (RBF) networks”. In: *Proceedings of 27th Asilomar conference on signals, systems and computers*. IEEE. 1993, pp. 401–405.
- [42] Kaitlin Gili et al. “Do quantum circuit born machines generalize?” In: *Quantum Science and Technology* 8.3 (2023), p. 035021.